

Milestone 1: Food Delivery (Talabat) Platform
Core Services Foundation

Deadline is Saturday 11/04/2026 at 11:59 PM

Contents

1 Overview	5
2 Personalization & Team Roster	5
2.1 team.json (Required)	5
2.2 Branch Naming (Mandatory)	6
2.3 Commit Message Format (Mandatory)	6
3 Project Structure — Multi-Module Maven	6
3.1 Port & Database Configuration	6
4 Step-by-Step: Creating the Multi-Module Project	7
4.1 Using IntelliJ IDEA (Recommended)	7
4.1.1 A1. Create the Parent Project	7
4.1.2 A2. Add Each Service as a Module	7
4.1.3 A3. What IntelliJ Does (and Does Not Do) Automatically	8
4.1.4 A4. Verify and Complete the Root POM	8
4.1.5 A5. Important: Do NOT Modify Child Service POMs	9
4.1.6 Add jackson-databind Dependency	9
4.1.7 Configure application.properties per Service	9
4.1.8 Create Sub-Packages	9
4.1.9 Add a Health Endpoint to Each Service	9
4.1.10 Create team.json	10
4.1.11 Verify the Build	10
4.2 Final Project Structure	10
5 Git Workflow — Initial Setup	11
6 Database Setup (Docker Compose)	11
7 Entity Models	11
7.1 User Service Entities (user-service)	12
7.1.1 User Entity	12
7.1.2 DeliveryAddress Entity	12

7.2	Restaurant Service Entities (restaurant-service)	12
7.2.1	Restaurant Entity	12
7.2.2	MenuItem Entity	13
7.3	Order Service Entities (order-service)	13
7.3.1	Order Entity	13
7.3.2	OrderItem Entity	14
7.4	Delivery Service Entities (delivery-service)	14
7.4.1	Delivery Entity	14
7.5	Checkout Service Entities (checkout-service)	15
7.5.1	Payment Entity	15
7.5.2	Offer Entity	15
7.5.3	PaymentOffer Entity (Join Entity)	15
7.6	Relationship Summary	16
7.7	Important: Cross-Service Data Access	16
8	Repository Layer	16
9	Features	17
9.1	User Service Features (port 8081)	17
9.1.1	[S1-F1] Search Users with Filters	17
9.1.2	[S1-F2] Update User Preferences (JSONB)	17
9.1.3	[S1-F3] Get User Order Summary (DTO)	18
9.1.4	[S1-F4] Deactivate User Account	18
9.1.5	[S1-F5] Filter Users by Preference (JSONB Query)	18
9.1.6	[S1-F6] Top Customers by Spending (Report DTO)	19
9.1.7	[S1-F7] Set Default Delivery Address (Transactional + Relationship)	19
9.1.8	[S1-F8] Get User Profile with Delivery Addresses (Relationship DTO)	19
9.1.9	[S1-F9] Find Users by Dietary Preference with Minimum Orders (JSONB + SQL)	20
9.2	Restaurant Service Features (port 8082)	20
9.2.1	[S2-F1] Search Restaurants by Cuisine and Rating Range	20
9.2.2	[S2-F2] Update Restaurant Details (JSONB Partial Update)	20
9.2.3	[S2-F3] Get Restaurant Order Revenue Summary (DTO)	21
9.2.4	[S2-F4] Update Restaurant Status (Transactional)	21
9.2.5	[S2-F5] Filter Restaurants by Detail Attribute (JSONB)	22
9.2.6	[S2-F6] Top Rated Restaurants Report (DTO)	22
9.2.7	[S2-F7] Rate a Restaurant After Order (Transactional)	22
9.2.8	[S2-F8] Toggle Menu Item Availability (Transactional + Relationship)	23
9.2.9	[S2-F9] Get Restaurants with Unavailable Menu Items (Relationship + Report DTO)	23
9.3	Order Service Features (port 8083)	23
9.3.1	[S3-F1] Get Orders by Status and Date Range	23
9.3.2	[S3-F2] Confirm Order and Assign Restaurant (Transactional)	24
9.3.3	[S3-F3] Get Order Cost Estimate (DTO)	24
9.3.4	[S3-F4] Deliver Order (Transactional Multi-Step)	25

9.3.5	[S3-F5] Filter Orders by Metadata Field (JSONB)	25
9.3.6	[S3-F6] Order Analytics by Time Period (Report DTO)	26
9.3.7	[S3-F7] Cancel Order (Transactional)	26
9.3.8	[S3-F8] Add Items to Existing Order (Transactional + Relationship)	26
9.3.9	[S3-F9] Get Order Details with Items (Relationship DTO)	27
9.4	Delivery Service Features (port 8084)	27
9.4.1	[S4-F1] Get Latest Delivery for an Order	27
9.4.2	[S4-F2] Create Delivery Record with Metadata (JSONB)	27
9.4.3	[S4-F3] Find Nearby Deliveries (DTO with Distance)	28
9.4.4	[S4-F4] Batch Delivery Updates (Transactional)	28
9.4.5	[S4-F5] Filter Deliveries by Metadata (JSONB Query)	28
9.4.6	[S4-F6] Order Delivery History in Date Range	29
9.4.7	[S4-F7] Purge Old Delivery Records (Transactional)	29
9.4.8	[S4-F8] Delivery Performance Summary (DTO)	29
9.4.9	[S4-F9] Find Delayed Deliveries (JSONB + Cross-Table DTO)	29
9.5	Checkout Service Features (port 8085)	30
9.5.1	[S5-F1] Get Payments by Status and Date Range	30
9.5.2	[S5-F2] Process Refund (Transactional + JSONB Update)	30
9.5.3	[S5-F3] User Payment Summary (DTO)	30
9.5.4	[S5-F4] Process Payment for Order (Transactional)	31
9.5.5	[S5-F5] Apply Offer to Payment (Transactional + Join Entity)	31
9.5.6	[S5-F6] Revenue Report by Date Range (Report DTO)	32
9.5.7	[S5-F7] Retry Failed Payment (Transactional)	33
9.5.8	[S5-F8] Get Payment Details with Applied Offers (Join Entity DTO)	34
9.5.9	[S5-F9] Get Most Used Offers Report (Join Entity + Aggregation)	34
10	Git Workflow — Feature Development	35
11	Dockerization (Phase D)	35
12	Work Distribution (Recommended):	36
13	Submission & Auto-Grader	36
14	Appendix: JSONB Sample Data	36
15	Appendix: Example Database Tables	38
15.1	User Service Tables	38
15.1.1	users table	38
15.1.2	delivery_addresses table	39
15.2	Restaurant Service Tables	39
15.2.1	restaurants table	39
15.2.2	menu_items table	40
15.3	Order Service Tables	41

15.3.1	<code>orders</code> table	41
15.3.2	<code>order_items</code> table	41
15.4	Delivery Service Tables	42
15.4.1	<code>deliveries</code> table	42
15.5	Checkout Service Tables	43
15.5.1	<code>payments</code> table	43
15.5.2	<code>offers</code> table	43
15.5.3	<code>payment_offers</code> table (Join Entity)	44
15.6	How the Tables Connect (Cross-Service)	44

1 Overview

This milestone is worth **15%** of your final grade. You will build a **Food Delivery Platform** consisting of 5 independently runnable Spring Boot services organized as a **Maven multi-module project**. All 5 services connect to a single shared PostgreSQL database and are orchestrated via a single `docker-compose.yaml`.

In future milestones, these services will be refactored into true microservices with separate databases, inter-service communication (OpenFeign, RabbitMQ, etc.), and Kubernetes deployment. For now, they are independent applications sharing one database.

Technologies: Spring Boot, PostgreSQL, Spring Data JPA, Docker, Docker Compose, Maven multi-module, Git & GitHub.

Services:

- a) **User Service** (port 8081) — Customer accounts, profiles, preferences
- b) **Restaurant Service** (port 8082) — Restaurant profiles, menus, ratings
- c) **Order Service** (port 8083) — Order lifecycle, line items, order history
- d) **Delivery Service** (port 8084) — Delivery tracking, driver assignments
- e) **Checkout Service** (port 8085) — Payment processing, refunds, promotions

Deliverables: 45 features (9 per service), full CRUD for all entities, Dockerized multi-service application, Git history with personalized feature branches and PRs.

2 Personalization & Team Roster

Every team member's work must be **individually identifiable** in Git history. The auto-grader cross-references the `team.json` roster against `git log` to determine who built what. Violations result in **zero credit** for the affected member.

2.1 team.json (Required)

Create a file named `team.json` in the **project root directory** (next to the parent `pom.xml`). It contains a JSON array with one entry per team member:

```
[{"studentId": "55-8078", "name": "Ahmed Ali", "githubUsername": "ahmed-ali-dev", "service": "user-service"}, ...]
```

Fields per entry:

- `studentId` — University ID in the format `XX-XXXX`
- `name` — Full name
- `githubUsername` — The exact GitHub username that will author commits
- `service` — Which service module this member belongs to (one of: `user-service`, `restaurant-service`, `order-service`, `delivery-service`, `checkout-service`)

The auto-grader reads this file and then runs specific tests to find each member's commits. It matches those commits against branch names to determine which features each member implemented. **Do not self-declare features** — the grader discovers them automatically.

2.2 Branch Naming (Mandatory)

All feature branches must include the implementing member's student ID:

```
feat/<service>/<feature-name>/<studentId>
```

Example: student 55-8078 implementing S1-F1 (Service 1, Feature 1) creates:

```
feat/user/S1-F1/55-8078
```

2.3 Commit Message Format (Mandatory)

All commits must follow this convention:

```
feat(<service name>): <description> (ID)
```

Examples: `feat(user-service): add User entity model (55-8078)`, `fix(order-service): fix null handling in search query (55-8078)`.

The auto-grader matches each commit's Git author (from `team.json`) against the student ID in the message. Mismatched or missing IDs will result in failures during grading. So even if you named the commit message wrong or the branch name was wrong and did follow the criteria, you know how to fix it using the commands that you took in the Git Lab.

3 Project Structure — Multi-Module Maven

The project is organized as a Maven multi-module project. The root directory contains a **parent POM** with `<packaging>pom</packaging>` that declares all 5 services as modules. Each service is a fully independent Spring Boot application with its own POM, source tree, and `@SpringBootApplication` entry point. Section 4 provides full step-by-step instructions for creating this structure.

3.1 Port & Database Configuration

All services connect to the **same PostgreSQL database** (`fooddeliverydb`). Each runs on a different port:

Service	Internal Port	Docker Host Port
user-service	8080	8081
restaurant-service	8080	8082
order-service	8080	8083
delivery-service	8080	8084
checkout-service	8080	8085

Each service's `application.properties` sets `server.port=8080`. The port differentiation happens in `docker-compose.yaml` via host-port mapping (e.g., `8081:8080` for `user-service`). When running services locally without Docker during development, each developer works on their own service and can use port 8080 since they only run one service at a time.

Each service configures: `spring.datasource.url=jdbc:postgresql://localhost:5432/fooddeliverydb` (or `postgres:5432` inside Docker), `ddl-auto=update`, and `show-sql=true`.

4 Step-by-Step: Creating the Multi-Module Project

This section walks you through creating the entire project structure from scratch. Two approaches are provided: **Option A** using IntelliJ IDEA (recommended), and **Option B** using Spring Initializr for students not using IntelliJ. Both produce the same result. Follow **one** of the two options, then continue with the shared steps.

Important architectural note: The root project acts as an **aggregator only**. It does not contain source code and does not act as a Maven parent for the child modules. Each service module keeps its own generated Spring Boot parent. The root `pom.xml` simply lists the modules so you can build them all with a single command.

4.1 Using IntelliJ IDEA (Recommended)

4.1.1 A1. Create the Parent Project

- a) Open IntelliJ IDEA → **File** → **New** → **Project**.
- b) On the left panel, select **Java**.
- c) On the right, set **Build system** to **Maven**.
- d) Set the **JDK** to **25**.
- e) Uncheck **Add sample code**.
- f) Expand **Advanced Settings** (at the bottom of the dialog).
- g) Fill in:
 - **Name:** `fooddelivery`
 - **GroupId:** `com.teamXX.fooddelivery` (replace `XX` with your team number)
 - **ArtifactId:** `fooddelivery`
- h) Click **Create**.
- i) IntelliJ creates a Maven project with a `pom.xml`. Open this `pom.xml` and manually add the following line inside the `<project>` block (after `<version>`):

`<packaging>pom</packaging>`
- j) If IntelliJ created a `src/` folder in the root project, **delete it** — the root aggregator project has no source code.
- k) **Reload Maven:** Click the Maven reload icon (circular arrows) in the Maven tool window, or right-click the `pom.xml` → **Maven** → **Reload project**.

4.1.2 A2. Add Each Service as a Module

Repeat the following for each of the 5 services (`user-service`, `restaurant-service`, `order-service`, `delivery-service`, `checkout-service`):

- a) Right-click the project root (`fooddelivery`) in the Project panel → **New** → **Module**.
- b) Select **Spring Boot** on the left.
- c) Configure the service information:
 - **Name / Artifact:** the service name (e.g., `user-service`)

- **Group:** `com.teamXX.fooddelivery`
- **Package name:** Use the service domain name **without hyphens**:
 - `com.teamXX.fooddelivery.user`
 - `com.teamXX.fooddelivery.restaurant`
 - `com.teamXX.fooddelivery.order`
 - `com.teamXX.fooddelivery.delivery`
 - `com.teamXX.fooddelivery.checkout`
- **Type / Build system:** Maven
- **Java:** 25
- **Packaging:** Jar

Warning: The `artifactId` can use hyphens (e.g., `user-service`), but **Java package names cannot contain hyphens**. Use `.user`, not `.user-service`, in the package name.

- Click **Next** to continue to the dependencies page.
- Select these dependencies: **Spring Web**, **Spring Boot DevTools**, **Spring Data JPA**, **PostgreSQL Driver**, **Rest Repositories**.
- Click **Create**.

4.1.3 A3. What IntelliJ Does (and Does Not Do) Automatically

After creating each service module, IntelliJ will usually:

- Create the service folder with its own `pom.xml` and `src/` tree.
- Generate the `@SpringBootApplication` main class.

However, IntelliJ may **not** automatically add the `<module>` entry to the root `pom.xml`. You must verify this yourself after each module creation.

4.1.4 A4. Verify and Complete the Root POM

After creating all 5 service modules:

- Open the root `fooddelivery/pom.xml`.
- Confirm that `<packaging>pom</packaging>` exists.
- Check whether all 5 services appear inside a `<modules>` block. If **any are missing**, add them manually:

```
<modules>
  <module>user-service</module>
  <module>restaurant-service</module>
  <module>order-service</module>
  <module>delivery-service</module>
  <module>checkout-service</module>
</modules>
```

- Save the file.
- Reload Maven** to pick up the changes.
- Confirm there are no Maven errors in the tool window.

4.1.5 A5. Important: Do NOT Modify Child Service POMs

Each generated service already has its own `<parent>` block pointing to Spring Boot. **Do not replace or modify this parent.** The root `fooddelivery` POM acts only as an **aggregator** (it lists the modules so you can build them all at once). It is **not** a Maven parent for the child modules.

In other words:

- **Do not** add a `<parent>` block pointing to `fooddelivery` inside any child POM.
- **Do not** remove the existing Spring Boot `<parent>` from any child POM.
- The only connection between the root POM and the children is the `<modules>` block in the root.

4.1.6 Add jackson-databind Dependency

In each service's `pom.xml`, add the following dependency inside the `<dependencies>` block (required for Hibernate JSONB support):

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>
```

After adding, **reload Maven**.

4.1.7 Configure application.properties per Service

Inside each service's `src/main/resources/application.properties`, add:

```
server.port=8080

spring.datasource.url=jdbc:postgresql://localhost:5432/fooddeliverydb
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=
    org.hibernate.dialect.PostgreSQLDialect
```

All services use port 8080 internally. The port differentiation to 8081–8085 happens in `docker-compose.yml` (covered in the Dockerization section).

4.1.8 Create Sub-Packages

Inside each service's main package (e.g., `src/main/java/com/teamXX/fooddelivery/user/`), create these sub-packages: `model/`, `repository/`, `service/`, `controller/`, `dto/`.

4.1.9 Add a Health Endpoint to Each Service

In each service's `controller/` package, create a simple health controller that exposes a GET endpoint returning "OK" (200):

- User Service: GET `/api/users/health`
- Restaurant Service: GET `/api/restaurants/health`
- Order Service: GET `/api/orders/health`

- Delivery Service: GET /api/deliveries/health
- Checkout Service: GET /api/payments/health

4.1.10 Create team.json

In the project root (next to the root `pom.xml`), create `team.json` as described in Section 2.

4.1.11 Verify the Build

Start PostgreSQL via Docker:

```
docker compose up -d
```

(This requires the `docker-compose.yaml` with just the PostgreSQL service — see Section 6.)

Then build all services from the project root:

```
mvn clean package -DskipTests
```

If the build succeeds, you should see `BUILD SUCCESS` and a JAR in each service's `target/` directory.

To test a single service locally:

```
cd user-service
mvn spring-boot:run
```

Then visit `http://localhost:8080/api/users/health` to confirm it responds with "OK".

4.2 Final Project Structure

After completing all steps, your project should look like this:

```
foodelivery/
+-- pom.xml                (root aggregator POM, packaging=pom)
+-- team.json              (team roster)
+-- docker-compose.yaml    (PostgreSQL, later: all services)
+-- .gitignore
+-- user-service/
|   +-- pom.xml            (own Spring Boot parent, NOT foodelivery)
|   +-- Dockerfile         (added in Phase D)
|   +-- src/main/java/com/teamXX/foodelivery/user/
|       +-- UserApplication.java    (@SpringBootApplication)
|       +-- model/
|       +-- repository/
|       +-- service/
|       +-- controller/
|       +-- dto/
+-- restaurant-service/
|   +-- (same structure)
+-- order-service/
|   +-- (same structure)
+-- delivery-service/
|   +-- (same structure)
+-- checkout-service/
|   +-- (same structure)
```

5 Git Workflow — Initial Setup

- a) Initialize a Git repository in the project root.
- b) Create a `.gitignore` excluding: `target/`, `.idea/`, `*.iml`, `.env`, `*.class`, `.DS_Store`.
- c) Create the parent POM, all 5 child POM files, the package structure, and the `team.json` file.
- d) Make an initial commit: `init: project setup by (Name_ID)` (e.g: `init: project setup by (Mohamed_Ayman_52_8078)`) (note: this is supposed to be the only commit message that will not follow the commit message rule that is stated in this description).
- e) Create a **private** GitHub repository named `TeamNumber-TeamName-FoodDelivery` (e.g: `40-Random-FoodDelivery`).
- f) Push to remote and rename the default branch to `main`.
- g) **Branch protection:** Enable “*Require a pull request before merging*” on `main`. (The rule may not be enforced on the free version of git so you have to enforce it manually as the grader will check the generated PRs)
- h) Add `Scalable-Submissions` as a collaborator.
- i) **Each member** must contribute to the project through Git using their own GitHub account. The auto-grader verifies all GitHub usernames from `team.json` appear in history of the repo. *If someone didn't contribute to the project (no-commits for him/her), will be considered as if they didn't work at all in the milestone and will receive a ZERO as a result in the milestone grade.*

6 Database Setup (Docker Compose)

The `docker-compose.yaml` in the project root contains a PostgreSQL service with:

- Container name: `fooddelivery-db`
- Port mapping: `5432:5432`
- Environment variables: user `postgres`, password `postgres`, database `fooddeliverydb`
- Named volume `pgdata` for data persistence

In Phase D (Dockerization), you will add all 5 application services to this same file.

Verify: Run `docker compose up -d` (PostgreSQL only at first), start any service locally, and confirm it connects to the database.

7 Entity Models

All entities use auto-generated `Long` IDs. Some entities include one JSONB column implemented as a `Map<String, Object>` using the Hibernate JSONB annotations learned in Lab 4. Since all services share one database, tables from different services coexist in the same database schema.

Each service defines JPA relationships (`@OneToMany`, `@ManyToOne`, join entities) **between entities within the same service only**. References to entities in other services are stored as plain `Long` foreign key columns (not JPA-managed relationships), and cross-service data access uses native SQL JOINS on the shared database. Remember to use `@JsonIgnore` on bidirectional relationships to prevent infinite recursion during JSON serialization.

7.1 User Service Entities (user-service)

7.1.1 User Entity

Table: users

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
name	String	not null	
email	String	not null, unique	
password	String	not null	
phone	String	not null, unique	
role	ENUM ¹	not null	"CUSTOMER" or "ADMIN"
status	ENUM	not null, default ACTIVE	ACTIVE / DEACTIVATED
preferences	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	Set on creation
deliveryAddresses	List<DeliveryAddress>	–	Relational Attribute

JSONB — **preferences**: Language, notification settings (email/sms booleans), default payment method, dietary restrictions (vegetarian, halal, etc.).

Relationships: *One* User can have *Many* Addresses and *User* is the inverse side and *DeliveryAddress* is the owner side.

7.1.2 DeliveryAddress Entity

Table: delivery_addresses

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
label	String	not null	e.g., "Home", "Office"
streetAddress	String	not null	Full street address
city	String	not null	
latitude	Double	not null	
longitude	Double	not null	
isDefault	Boolean	default false	
metadata	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	
user	User	–	Relational Attribute

JSONB — **metadata**: Delivery instructions, gate code, floor number, landmark, phone for delivery.

Relationships: *Many* Addresses can belong to *One* User and *User* is the inverse side and *DeliveryAddress* is the owner side.

7.2 Restaurant Service Entities (restaurant-service)

7.2.1 Restaurant Entity

Table: restaurants

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
name	String	not null	
email	String	not null, unique	
phone	String	not null, unique	
cuisineType	ENUM ²	not null	See values below
status	ENUM	not null	OPEN / CLOSED / SUSPENDED
rating	Double	default 0.0	Running average
totalRatings	Integer	default 0	Count for average calculation
details	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	
menuItems	List<MenuItem>	–	Relational Attribute

Cuisine types: EGYPTIAN, ITALIAN, CHINESE, FAST_FOOD, SEAFOOD.

JSONB — details: Address, opening hours, delivery radius (km), minimum order amount, average preparation time (minutes), license number.

Relationships: *One* Restaurant has *Many* MenuItems and **Restaurant** is the inverse side and **MenuItem** is the owner side.

7.2.2 MenuItem Entity

Table: menu_items

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
name	String	not null	
description	String	not null	
price	Double	not null	
category	ENUM ³	not null	APPETIZER / MAIN / DESSERT / DRINK
available	Boolean	default true	
metadata	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	
restaurant	Restaurant	–	Relational Attribute

JSONB — metadata: Calories, allergens (list), preparation time (minutes), spice level, dietary tags (vegetarian, vegan, gluten-free).

Relationships: *Many* MenuItems belong to *One* Restaurant and **Restaurant** is the inverse side and **MenuItem** is the owner side.

7.3 Order Service Entities (order-service)

7.3.1 Order Entity

Table: orders

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
userId	Long	not null	FK reference (not JPA-managed)
restaurantId	Long	nullable	FK reference (assigned later)
deliveryAddress	String	not null	
status	ENUM ⁴	not null	See values below
totalAmount	Double	nullable	Set when calculated
metadata	Map<String, Object>	JSONB	See below
orderDate	LocalDateTime	not null	
deliveredAt	LocalDateTime	nullable	Set when delivered
orderItems	List<OrderItem>	–	Relational Attribute

Status values: PLACED, CONFIRMED, PREPARING, DELIVERED, CANCELLED.

JSONB — metadata: Special instructions (no onions, extra sauce, etc.), cutlery requested (boolean), estimated delivery time (minutes), delivery type (standard, express).

Note: `userId` and `restaurantId` are plain Long columns, **not** JPA @ManyToOne relationships (User and Restaurant live in different services).

Relationships: *One* Order can have *Many* Items and **Order** is the inverse side while **OrderItem** is the owner side.

7.3.2 OrderItem Entity

Table: order_items

An order can contain multiple items (e.g., ordering a burger, fries, and a drink). Each item has a line number within the order.

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
lineNumber	Integer	not null	1, 2, 3...
menuItemId	Long	not null	FK reference (not JPA-managed)
itemName	String	not null	Snapshot at order time
quantity	Integer	not null	
unitPrice	Double	not null	Price at order time
status	ENUM ⁵	not null	PENDING / PREPARED / CANCELLED
metadata	Map<String, Object>	JSONB	See below
order	Order	–	Relational Attribute

JSONB — metadata: Customizations (e.g., extra cheese, no pickles), size selected, notes.

Relationships: *Many* Items can belong to *One* Order and **Order** is the inverse side while **OrderItem** is the owner side.

7.4 Delivery Service Entities (delivery-service)

7.4.1 Delivery Entity

Table: deliveries

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
orderId	Long	not null	FK reference (not JPA-managed)
driverName	String	not null	
latitude	Double	not null	Current driver position
longitude	Double	not null	
status	ENUM ⁶	not null	ASSIGNED / PICKED_UP / IN_TRANSIT / DELIVERED
updatedAt	LocalDateTime	not null	
metadata	Map<String, Object>	JSONB	See below

JSONB — **metadata**: Estimated arrival (minutes), distance remaining (km), vehicle type, speed (km/h), contact phone.

Relationships: None within this service. `orderId` is a plain `Long` referencing the `orders` table in the shared database via native SQL.

7.5 Checkout Service Entities (checkout-service)

This service demonstrates the **join entity pattern** for many-to-many relationships with extra columns (like the `OrderItem` pattern from Lab 4).

7.5.1 Payment Entity

Table: payments

Field	Type	Constraints	Notes
<code>id</code>	<code>Long</code>	PK, auto-generated	
<code>orderId</code>	<code>Long</code>	not null	FK reference (not JPA-managed)
<code>userId</code>	<code>Long</code>	not null	FK reference (not JPA-managed)
<code>amount</code>	<code>Double</code>	not null	
<code>method</code>	<code>ENUM</code> ⁷	not null	CREDIT_CARD / CASH_ON_DELIVERY / WALLET
<code>status</code>	<code>ENUM</code>	not null	See below
<code>transactionDetails</code>	<code>Map<String, Object></code>	JSONB	See below
<code>createdAt</code>	<code>LocalDateTime</code>	not null	
<code>paymentOffers</code>	<code>List<PaymentOffer></code>	–	Relational Attribute

Status values: PENDING, COMPLETED, FAILED, REFUNDED.

JSONB — **transactionDetails**: Gateway response, card last four digits, receipt URL, failure reason, tip amount.

Relationships: *One* `Payment` can reference *Many* `PaymentOffers` and *Payment* is the inverse side while *PaymentOffer* is the owner side.

7.5.2 Offer Entity

Table: offers

Field	Type	Constraints	Notes
<code>id</code>	<code>Long</code>	PK, auto-generated	
<code>code</code>	<code>String</code>	not null, unique	e.g., "HUNGRY30"
<code>discountType</code>	<code>ENUM</code> ⁸	not null	PERCENTAGE or FIXED
<code>discountValue</code>	<code>Double</code>	not null	e.g., 30.0
<code>maxUses</code>	<code>Integer</code>	not null	
<code>currentUses</code>	<code>Integer</code>	default 0	
<code>expiryDate</code>	<code>LocalDateTime</code>	not null	
<code>active</code>	<code>Boolean</code>	default true	
<code>metadata</code>	<code>Map<String, Object></code>	JSONB	See below
<code>paymentOffers</code>	<code>List<PaymentOffer></code>	–	Relational Attribute

JSONB — **metadata**: Eligible cuisine types, minimum order amount, terms and conditions, applicable restaurants.

Relationships: *One* `Offer` can reference *Many* `PaymentOffers` and *Offer* is the inverse side while *PaymentOffer* is the owner side.

7.5.3 PaymentOffer Entity (Join Entity)

Table: payment_offers

This is a **join entity** linking **Payment** and **Offer** in a many-to-many relationship with extra columns — the same pattern as **OrderItem** from Lab 4. A payment can use multiple offers, and an offer can be used on multiple payments.

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
discountApplied	Double	not null	Actual amount deducted
appliedAt	LocalDateTime	not null	When offer was applied
payment	Payment	–	Relational Attribute
offer	Offer	–	Relational Attribute

Relationships:

- *Many* PaymentOffers can belong to *One* Payment and **Payment** is the inverse side while **PaymentOffer** is the owner side.
- *Many* PaymentOffers can belong to *One* Offer and **Offer** is the inverse side while **PaymentOffer** is the owner side.

7.6 Relationship Summary

Service	Relationship	Pattern Tested
User	User ↔ DeliveryAddress	Bidirectional
Restaurant	Restaurant ↔ MenuItem	Bidirectional
Order	Order ↔ OrderItem	Bidirectional + ordering ⁹
Delivery	(no JPA relations)	Cross-service via native SQL
Checkout	Payment ↔ Offer	Join entity (PaymentOffer)

7.7 Important: Cross-Service Data Access

Since all services share one PostgreSQL database, any service can query any table using **native SQL**. For example, the Order Service can JOIN `orders` with `restaurants` or `payments` in a native `@Query`. However, each service only defines JPA entity classes and JPA relationships for entities within its *own* module. Cross-table access is always via native SQL, never via JPA relationships. This pattern will be replaced by Micro-services Communications in Milestone 3.

8 Repository Layer

Each service defines a `JpaRepository` for **each of its entities**. For services with multiple entities, you need multiple repository interfaces. ***Make sure that you make use of the Repository Layer when interacting with the database and not have the queries at the service layer as the auto grader will test having proper layering logic in your code.*** Repositories include:

- **Naming-convention methods** for simple lookups.
- **Custom @Query methods with native SQL** for complex queries, including JOINS against other services' tables in the shared database.
- **@Modifying + @Transactional** for UPDATE/DELETE queries.

9 Features

CRUD operations (create, read by ID, read all, update, delete) are the **baseline** for **every entity in every service** and are NOT counted as features. *However this doesn't mean that you don't need to implement it as it will have test cases and the auto grader will make use of it and will not run if they are not there so* This means: User Service needs CRUD for both User and DeliveryAddress, Restaurant Service for both Restaurant and MenuItem, Order Service for both Order and OrderItem, Delivery Service for Delivery, and Checkout Service for Payment, Offer, and PaymentOffer. Implement all CRUD first before starting features.

Important Note: *Do not forget to apply the Layered architecture (having repository, service and controller) as the test case will test that each feature is implemented according to the layered architecture as explained to you in the Labs.*

9.1 User Service Features (port 8081)

9.1.1 [S1-F1] Search Users with Filters

Branch: feat/user/S1-F1/<ID>

Endpoint: GET /api/users/search?name={name}&email={email}&role={role}

Response: List of users matching any non-null filter. Name and email support partial matching (case-insensitive). Return empty list if no matches.

Note: You may use Native SQL query but you will have to find a way to handle null parameters.

Test scenario:

- a) Create 3 users via CRUD: one with name “Ahmed”, role “CUSTOMER”; one with name “Sara”, role “ADMIN”; one with name “Ahmed Ali”, role “CUSTOMER”.
- b) GET /api/users/search?name=Ahmed → should return 2 users.
- c) GET /api/users/search?role=ADMIN → should return only “Sara”.
- d) GET /api/users/search?name=xyz → should return empty list.
- e) GET /api/users/search?name=ahmed → should still return 2 (case-insensitive).

9.1.2 [S1-F2] Update User Preferences (JSONB)

Branch: feat/user/S1-F2/<ID>

Endpoint: PUT /api/users/{id}/preferences

Request body: JSON object of preference key-value pairs.

Behavior: Find user —*throws 404 if user not found*. Merge incoming preferences into existing map—do not overwrite. The only case that you overwrite the preferences is that if the same keys are already there in the preferences, otherwise if it's a new preference then add it to the current preferences. Return updated user.

Test scenario:

- a) Create a user with preferences {"language": "en", "dietaryRestrictions": "none"}.
- b) PUT /api/users/{id}/preferences with body {"dietaryRestrictions": "vegetarian", "defaultPayment": "WALLET"}
- c) Verify response has {"language": "en", "dietaryRestrictions": "vegetarian", "defaultPayment": "WALLET"} — “language” kept, “dietaryRestrictions” updated, “defaultPayment” added.
- d) PUT /api/users/999/preferences → should return 404.

9.1.3 [S1-F3] Get User Order Summary (DTO)

Branch: feat/user/S1-F3/<ID>

Endpoint: GET /api/users/{id}/order-summary

Response DTO (UserOrderSummaryDTO): userId, name, totalOrders, deliveredOrders, cancelledOrders, totalSpent, averageOrderAmount.

Behavior: Find user *–throws 404 if user not found*. Aggregate data from the orders table for this user using native SQL JOINS on the shared database. Build and return DTO.¹⁰

Test scenario:

- a) Create a user. Create 5 orders: 3 DELIVERED with amounts 80, 120, 200; 1 CANCELLED; 1 PLACED.
- b) GET /api/users/{id}/order-summary → should return: totalOrders=5, deliveredOrders=3, cancelledOrders=1, totalSpent=400, averageOrderAmount=133.33.
- c) Test with a user who has no orders → all values should be 0.
- d) Test with non-existent user ID → 404.

9.1.4 [S1-F4] Deactivate User Account

Branch: feat/user/S1-F4/<ID>

Endpoint: PUT /api/users/{id}/deactivate

Behavior (Transactional): Find user *–throws 404 if user not found*. Check no active orders exist for this user in the orders table *–throw 400 if the user has active orders*. Set status to "DEACTIVATED". Save.

Test scenario:

- a) Create a user. Create an order with status PLACED for that user.
- b) PUT /api/users/{id}/deactivate → should return 400 (active order exists).
- c) Update the order status to DELIVERED.
- d) PUT /api/users/{id}/deactivate → should succeed. Verify user status is now "DEACTIVATED".

9.1.5 [S1-F5] Filter Users by Preference (JSONB Query)

Branch: feat/user/S1-F5/<ID>

Endpoint: GET /api/users/preferences/search?key={key}&value={value}

Behavior: Validate key/value not blank *–throws 400 if they are*. Return users whose preferences matches key-value pair.

Test scenario:

- a) Create 3 users: one with preferences {"dietaryRestrictions":"vegetarian"}, one with {"dietaryRestrictions":"halal"}, one with {"dietaryRestrictions":"vegetarian"}.
- b) GET /api/users/preferences/search?key=dietaryRestrictions&value=vegetarian → should return 2 users.
- c) GET /api/users/preferences/search?key=dietaryRestrictions&value=vegan → empty list.
- d) GET /api/users/preferences/search?key=&value=vegetarian → should return 400.

¹⁰Since you will use native SQL here, you can construct the DTO manually from the Object[] that will be returned from the query

9.1.6 [S1-F6] Top Customers by Spending (Report DTO)

Branch: feat/user/S1-F6/<ID>

Endpoint: GET /api/users/reports/top-customers?startDate={date}&endDate={date}&limit={n}

Response DTO (TopCustomerDTO): userId, name, totalSpent, orderCount.

Behavior: The feature means show me the top N customers who spent the most money on food orders in a given time period. Validate dates —*throws 400 if not valid (if start after end)*. Build the query and construct a DTO from the result of the query in the same way as you did in S1-F3.

Test scenario:

- a) Create 3 users. Create delivered orders: User A = 800 total, User B = 1500 total, User C = 300 total, all within March 2026.
- b) GET /api/users/reports/top-customers?startDate=2026-03-01&endDate=2026-03-31&limit=2 → should return User B first, then User A.
- c) Test with dates where no orders exist → empty list.
- d) Test with startDate after endDate → 400.

9.1.7 [S1-F7] Set Default Delivery Address (Transactional + Relationship)

Branch: feat/user/S1-F7/<ID>

Endpoint: PUT /api/users/{userId}/addresses/{addressId}/default

Behavior (Transactional): Find user —*throws 404 if user not found*. Find the DeliveryAddress by ID and verify it belongs to this user —*throw 404 if user not found or the address not found and 400 if the passed address doesn't belong to that user*. Set isDefault = false on all of the user's existing addresses. Set isDefault = true on the target address. Return the updated user with their addresses.

Test scenario:

- a) Create a user with 3 delivery addresses. Set address #2 as default.
- b) PUT /api/users/{userId}/addresses/{addressId}/default → verify address #3 is now default and #2 is no longer default.
- c) Test with non-existent user → 404.
- d) Test with an address ID that belongs to a different user → 400.

9.1.8 [S1-F8] Get User Profile with Delivery Addresses (Relationship DTO)

Branch: feat/user/S1-F8/<ID>

Endpoint: GET /api/users/{id}/profile

Response DTO (UserProfileDTO): userId, name, email, phone, preferences (JSONB), deliveryAddresses (list of address objects with label, streetAddress, city, lat/lng, isDefault, meta-data), totalAddresses (count).

Behavior: Find user with their delivery addresses loaded —*throws (404 if user not found)*. Build the DTO from the user entity and its delivery address collection. The DTO must include the full list of delivery addresses, not just IDs.

Test scenario:

- a) Create a user with preferences and 3 delivery addresses (one marked default).
- b) GET /api/users/{id}/profile → verify DTO contains user info, preferences, full address list with all fields, and totalAddresses = 3.
- c) Create a user with no addresses → verify deliveryAddresses is empty list and totalAddresses = 0.
- d) Test with non-existent user → 404.

9.1.9 [S1-F9] Find Users by Dietary Preference with Minimum Orders (JSONB + SQL)

Branch: feat/user/S1-F9/<ID>

Endpoint: GET /api/users/preferences/dietary?diet={diet}&minOrders={n}

Behavior: This feature is as if you want to find me all users who have dietary restriction x and have placed at least n delivered orders. Validate diet not blank *—throws 400 if blank*.

Test scenario:

- Create 3 users: User A (dietaryRestrictions=vegetarian, 5 delivered orders), User B (dietaryRestrictions=vegetarian, 2 delivered orders), User C (dietaryRestrictions=halal, 10 delivered orders).
- GET /api/users/preferences/dietary?diet=vegetarian&minOrders=3 → should return only User A.
- GET /api/users/preferences/dietary?diet=vegetarian&minOrders=1 → should return User A and User B.
- GET /api/users/preferences/dietary?diet=&minOrders=1 → 400.

9.2 Restaurant Service Features (port 8082)

9.2.1 [S2-F1] Search Restaurants by Cuisine and Rating Range

Branch: feat/restaurant/S2-F1/<ID>

Endpoint: GET /api/restaurants/search?cuisineType={c}&minRating={r}&maxRating={r}

Behavior: Validate range *—throws 400 if not valid (for example if min > max)*. Returns a list of restaurants that match the search criteria ordered by rating descending.

Test scenario:

- Create 3 restaurants: rating 4.5 (EGYPTIAN, OPEN), rating 3.8 (ITALIAN, CLOSED), rating 4.9 (EGYPTIAN, OPEN).
- GET /api/restaurants/search?cuisineType=EGYPTIAN&minRating=4.0&maxRating=5.0 → should return 2, 4.9 first.
- GET /api/restaurants/search?minRating=3.0&maxRating=4.0 → returns the 3.8 restaurant (no cuisine filter).
- GET /api/restaurants/search?minRating=5.0&maxRating=3.0 → 400.

9.2.2 [S2-F2] Update Restaurant Details (JSONB Partial Update)

Branch: feat/restaurant/S2-F2/<ID>

Endpoint: PUT /api/restaurants/{id}/details

Request body: JSON object of detail fields to update.

Behavior: Find restaurant *—throws 404 if not found*. Merge incoming fields into existing **details** map. meaning if different attributes were sent in the json body, then add it to the existing one, else then you will update it. For example, if the restaurant's current details is:

```
{"address": "Zamalek, Cairo", "deliveryRadius": 5, "minOrder": 50}
```

and the request body sends:

```
{"deliveryRadius": 10, "openingHours": "9AM-11PM"}
```

The result should be:

```
{"address": "Zamalek, Cairo", "deliveryRadius": 10, "minOrder": 50, "openingHours": "9AM-11PM"}
```

Save the updated data and return the updated restaurant record.

Test scenario:

- a) Create a restaurant with details {"address":"Zamalek","deliveryRadius":5,"minOrder":50}.
- b) PUT /api/restaurants/{id}/details with body {"deliveryRadius":10,"openingHours":"9AM-11PM"}.
- c) Verify response: address=Zamalek (kept), deliveryRadius=10 (updated), minOrder=50 (kept), openingHours=9AM-11PM (added).
- d) Test with non-existent restaurant → 404.

9.2.3 [S2-F3] Get Restaurant Order Revenue Summary (DTO)

Branch: feat/restaurant/S2-F3/<ID>

Endpoint: GET /api/restaurants/{id}/revenue?startDate={date}&endDate={date}

Response DTO (RestaurantRevenueDTO): restaurantId, name, totalOrders, totalRevenue, averageOrderAmount.

Behavior: Find restaurant —*throws 404 if not found*. Aggregate on orders table for this restaurant in date range and return the required DTO.

Test scenario:

- a) Create a restaurant. Create 5 delivered orders for this restaurant in March with amounts 100, 150, 200, 250, 300.
- b) GET /api/restaurants/{id}/revenue?startDate=2026-03-01&endDate=2026-03-31 → totalOrders=5, totalRevenue=1000, averageOrderAmount=200.
- c) Test with date range where no orders exist → all zeroes.
- d) Test with non-existent restaurant → 404.

9.2.4 [S2-F4] Update Restaurant Status (Transactional)

Branch: feat/restaurant/S2-F4/<ID>

Endpoint: PUT /api/restaurants/{id}/status

Request body: status.

Behavior (Transactional): Find restaurant —*throws 404 if not found*. If going SUSPENDED, check no active orders reference this restaurant —*throws 400 if not*. Update status and save. No need to return it just *return status code 200*.

Test scenario:

- a) Create a restaurant (OPEN). Create a CONFIRMED order for it.
- b) PUT /api/restaurants/{id}/status with {"status":"SUSPENDED"} → 400 (active order exists).
- c) Deliver the order. Now try SUSPENDED again → 200.
- d) Verify restaurant status is SUSPENDED.

9.2.5 [S2-F5] Filter Restaurants by Detail Attribute (JSONB)

Branch: feat/restaurant/S2-F5/<ID>

Endpoint: GET /api/restaurants/details/search?key={key}&value={value}&status={status}

Behavior: Filter restaurants by a JSONB details key-value pair and optional status (the status is not required and means if the restaurant is OPEN, CLOSED, or SUSPENDED).

Test scenario:

- a) Create 3 restaurants: one with details {"deliveryRadius":10} (OPEN), one with {"deliveryRadius":5} (OPEN), one with {"deliveryRadius":10} (CLOSED).
- b) GET /api/restaurants/details/search?key=deliveryRadius&value=10&status=OPEN → returns 1.
- c) GET /api/restaurants/details/search?key=deliveryRadius&value=10 → returns 2 (no status filter).
- d) GET /api/restaurants/details/search?key=deliveryRadius&value=20 → empty list.

9.2.6 [S2-F6] Top Rated Restaurants Report (DTO)

Branch: feat/restaurant/S2-F6/<ID>

Endpoint: GET /api/restaurants/reports/top-rated?limit={n}

Response DTO (TopRestaurantDTO): restaurantId, name, rating, totalOrders.

Behavior: Get the top rated (n) restaurants and return them as TopRestaurantDTO.

Test scenario:

- a) Create 3 restaurants with ratings 4.9, 4.5, 4.2. Create some delivered orders for each.
- b) GET /api/restaurants/reports/top-rated?limit=2 → returns 2, 4.9 first.
- c) GET /api/restaurants/reports/top-rated?limit=10 → returns all 3.

9.2.7 [S2-F7] Rate a Restaurant After Order (Transactional)

Branch: feat/restaurant/S2-F7/<ID>

Endpoint: POST /api/restaurants/{id}/rate

Request body: orderId and rating (1–5).

Behavior (Transactional): Find restaurant —*throws 404 if not found*. Verify order exists and references this restaurant and is DELIVERED —*throws 400 if the order doesn't reference this restaurant or it's not delivered or the rate is not between 1 and 5 and throws 404 if the order is not found*. Recalculate running average. Update restaurant and Save. Return status *code 200*.

Test scenario:

- a) Create a restaurant (rating=0, totalRatings=0). Create a DELIVERED order for this restaurant.
- b) POST /api/restaurants/{id}/rate with {"orderId":1,"rating":5} → 200. Verify rating=5.0, totalRatings=1.
- c) Rate again with a different delivered order, rating=3 → verify rating=4.0, totalRatings=2.
- d) Test with rating=6 → 400. Test with non-delivered order → 400. Test with order for another restaurant → 400.

9.2.8 [S2-F8] Toggle Menu Item Availability (Transactional + Relationship)

Branch: feat/restaurant/S2-F8/<ID>

Endpoint: PUT /api/restaurants/{restaurantId}/menu-items/{menuItemId}/toggle

Request body: *toggledBy*.

Behavior (Transactional): Find restaurant *—throws 404 if not found*. Find the MenuItem by ID and verify it belongs to this restaurant *—throws 404 if menu item not found* and *—throws 400 if it doesn't belong to the specified restaurant*. If setting to unavailable (available = false), check no pending order items reference this menu item *—throws 400 if found*. Toggle the available field. Update the item's JSONB metadata to add toggledAt timestamp and toggledBy (from request body). In addition, verify the toggledBy that is the userID being sent in the request body is an actual **Admin** User *—(throw 403 if not)*. Return the updated restaurant with all menu items.

Test scenario:

- a) Create a restaurant. Add a MenuItem (available=true). Create an ADMIN user.
- b) PUT /api/restaurants/{restId}/menu-items/{itemId}/toggle with {"toggledBy":3} → item now available=false, metadata has toggledAt and toggledBy.
- c) Test with pending orders referencing this item → 400 when trying to set unavailable.
- d) Test with non-ADMIN toggler → 403. Test with item from different restaurant → 400.

9.2.9 [S2-F9] Get Restaurants with Unavailable Menu Items (Relationship + Report DTO)

Branch: feat/restaurant/S2-F9/<ID>

Endpoint: GET /api/restaurants/menu-items/unavailable

Response DTO (RestaurantMenuAlertDTO): restaurantId, restaurantName, restaurantStatus, unavailableItems (list of menu items where available = false), unavailableCount.

Behavior: Find all restaurants that have at least one unavailable menu item. For each matching restaurant, load their unavailable items. Build list of DTOs. Only include restaurants that match the criteria.

Test scenario:

- a) Create 3 restaurants. Add menu items: Rest A has 1 unavailable + 2 available, Rest B has all available, Rest C has 3 unavailable.
- b) GET /api/restaurants/menu-items/unavailable → returns 2 DTOs (Rest A with unavailableCount=1, Rest C with unavailableCount=3). Rest B not included.
- c) Test with all items available → empty list.

9.3 Order Service Features (port 8083)

9.3.1 [S3-F1] Get Orders by Status and Date Range

Branch: feat/order/S3-F1/<ID>

Endpoint: GET /api/orders/search?status={s}&startDate={d}&endDate={d}

Behavior: Query with date range on orderDate and optional status of the order. Order by most recent.

Test scenario:

- a) Create 5 orders: 2 DELIVERED in March, 1 PLACED in March, 2 DELIVERED in February.
- b) GET /api/orders/search?status=DELIVERED&startDate=2026-03-01&endDate=2026-03-31 → returns 2, most recent first.
- c) GET /api/orders/search?startDate=2026-03-01&endDate=2026-03-31 → returns 3 (no status filter).

9.3.2 [S3-F2] Confirm Order and Assign Restaurant (Transactional)

Branch: feat/order/S3-F2/<ID>

Endpoint: PUT /api/orders/{orderId}/confirm?restaurantId={restaurantId}

Behavior (Transactional): Find order — *throws 404 if not found*. Validate order status is PLACED (*—throws 400 if not valid*—can only confirm a newly placed order). Verify the restaurant exists and has status OPEN (*throws 404 if not found, and 400 if not open*). Set the order's restaurantId to the given restaurant. Set order status to CONFIRMED. Save order. Return the updated order.

Test scenario:

- a) Create a PLACED order (no restaurant). Create an OPEN restaurant.
- b) PUT /api/orders/{orderId}/confirm?restaurantId={restId} → order status=CONFIRMED, restaurantId set.
- c) Try confirming again → 400 (order not PLACED anymore).
- d) Try confirming with a CLOSED restaurant → 400.
- e) Try with non-existent order → 404. Non-existent restaurant → 404.

9.3.3 [S3-F3] Get Order Cost Estimate (DTO)

Branch: feat/order/S3-F3/<ID>

Endpoint: POST /api/orders/estimate

Request body: restaurantId, itemCount, deliveryDistance (km).

Response DTO (OrderCostEstimatedDTO): estimatedFoodCost, deliveryFee, serviceFee, estimatedTotal, surgeMultiplier.

Scenario: Before placing an order, a customer wants to see an estimated total — just like when food delivery apps show “Total: EGP 180 (incl. delivery)” before checkout. This endpoint computes that estimate. It does **not** create an order in the database — it only calculates and returns the result.

Behavior — step by step:

- a) **Calculate estimated food cost:** Look up the restaurant's average menu item price from menu_items table. Multiply by itemCount:

$$\text{foodCost} = \text{avgMenuPrice} * \text{itemCount}$$

For example, if average price is 60 EGP and itemCount is 3: $60 * 3 = 180$ EGP.

- b) **Calculate delivery fee:** Use a base rate of 10 EGP per km:

$$\text{deliveryFee} = 10.0 * \text{deliveryDistance}$$

For the same example with 5 km: $10 * 5 = 50$ EGP.

- c) **Calculate service fee:** Fixed at 5% of food cost:

$$\text{serviceFee} = \text{foodCost} * 0.05$$

For the example: $180 * 0.05 = 9$ EGP.

- d) **Determine surge multiplier:** This simulates how prices increase during peak hours. Count how many active orders (status IN PLACED, CONFIRMED, PREPARING) reference the same restaurant using a native SQL query.

Based on the count, apply the following surge tiers:

- 0–10 active orders → surgeMultiplier = 1.0 (normal)

- 11–20 active orders → `surgeMultiplier` = 1.2 (busy)
- More than 20 active orders → `surgeMultiplier` = 1.5 (peak hours)

e) **Calculate total:**

`total = (foodCost + deliveryFee + serviceFee) * surgeMultiplier`

For the example with no surge: $(180 + 50 + 9) * 1.0 = 239$ EGP.

Same order during peak hours: $(180 + 50 + 9) * 1.5 = 358.5$ EGP.

f) **Build and return the DTO** containing all five values.

Test scenario:

- Create a restaurant with menu items averaging 60 EGP.
- POST `/api/orders/estimate` with `restaurantId=1`, `itemCount=3`, `deliveryDistance=5`.
- Verify `foodCost=180`, `deliveryFee=50`, `serviceFee=9`, `surgeMultiplier=1.0`, `total=239`.
- Create 15 active orders for the same restaurant. Call estimate again → `surgeMultiplier` should be 1.2.
- Verify no order was created in the database (this endpoint is read-only).

9.3.4 [S3-F4] Deliver Order (Transactional Multi-Step)

Branch: `feat/order/S3-F4/<ID>`

Endpoint: PUT `/api/orders/{id}/deliver`

Behavior (Transactional): Find order –*throws 404 if not found*. Validate status PREPARING –*throws 400 if not*. Set DELIVERED + `deliveredAt`. Calculate `totalAmount` if unset (sum of `OrderItem` quantities * `unitPrices`). Create payment record with status PENDING. Save order. Return the order after the update.

Test scenario:

- Create an order with status PREPARING and 3 `OrderItems` (quantities 2, 1, 3 at prices 50, 80, 30).
- PUT `/api/orders/{id}/deliver` → order status=DELIVERED, `deliveredAt` set, `totalAmount=270`, a new PENDING payment created.
- Try delivering again → 400 (already DELIVERED).
- Try delivering a PLACED order → 400.

9.3.5 [S3-F5] Filter Orders by Metadata Field (JSONB)

Branch: `feat/order/S3-F5/<ID>`

Endpoint: GET `/api/orders/metadata/search?key={key}&value={value}`

Behavior: Validate key/value –*throws 400 if not valid*. Return matches.

Test scenario:

- Create 3 orders: one with metadata `{"deliveryType":"express"}`, two with `{"deliveryType":"standard"}`.
- GET `/api/orders/metadata/search?key=deliveryType&value=express` → returns 1 order.
- GET `/api/orders/metadata/search?key=&value=x` → 400.

9.3.6 [S3-F6] Order Analytics by Time Period (Report DTO)

Branch: feat/order/S3-F6/<ID>

Endpoint: GET /api/orders/analytics?startDate={d}&endDate={d}

Response DTO (OrderAnalyticsDTO): totalOrders, deliveredOrders, cancelledOrders, totalRevenue, averageOrderAmount, deliveryRate.

Behavior: Calculate the required fields for orders from the start date till the end date. Build the DTO and return it.

Test scenario:

- a) Create 10 orders in March: 7 DELIVERED (amounts: 80,100,120,140,160,180,200), 3 CANCELLED.
- b) GET /api/orders/analytics?startDate=2026-03-01&endDate=2026-03-31 → totalOrders=10, deliveredOrders=7, cancelledOrders=3, totalRevenue=980, averageOrderAmount=140, deliveryRate=70.0.

9.3.7 [S3-F7] Cancel Order (Transactional)

Branch: feat/order/S3-F7/<ID>

Endpoint: PUT /api/orders/{id}/cancel

Behavior (Transactional): Find order *–throws 404 if not found.* Validate status PLACED/CONFIRMED *–throws 400 if not valid.* Set CANCELLED. Cancel all PENDING delivery records for this order. Save and *return status code 200* if the workflow was done successfully.

Test scenario:

- a) Create a CONFIRMED order with a restaurant assigned and a PENDING delivery.
- b) PUT /api/orders/{id}/cancel → order status=CANCELLED.
- c) Try cancelling a DELIVERED order → 400.
- d) Try cancelling a PREPARING order → 400.

9.3.8 [S3-F8] Add Items to Existing Order (Transactional + Relationship)

Branch: feat/order/S3-F8/<ID>

Endpoint: POST /api/orders/{orderId}/items

Request body: A list of item objects, each with menuItemId, itemName, quantity, unitPrice, and metadata.

Behavior (Transactional): Find order *–throws 404 if not found.* Validate order status is PLACED or CONFIRMED (*–throws 400 if not valid*—cannot add items to orders being prepared or delivered). Validate each item has menuItemId, itemName, quantity, and unitPrice *–throws 400 if not valid.* Determine the lineNumber for each new item by continuing from the order's current max line number (or starting at 1 if no items exist). Create OrderItem entities with status = PENDING and add them to the order. Save the order. Return the updated order with all its items, ordered by lineNumber.

Test scenario:

- a) Create a PLACED order with no items.
- b) POST /api/orders/{orderId}/items with 2 items → verify items created with lineNumber 1 and 2, status=PENDING.
- c) Add 1 more item → verify lineNumber=3.
- d) Try adding items to a DELIVERED order → 400.
- e) Try adding an item with missing itemName → 400.

9.3.9 [S3-F9] Get Order Details with Items (Relationship DTO)

Branch: feat/order/S3-F9/<ID>

Endpoint: GET /api/orders/{orderId}/details

Response DTO (OrderDetailsDTO): orderId, userId, restaurantId, status, totalAmount, metadata (JSONB), items (list of item objects ordered by lineNumber, each with id, lineNumber, itemName, quantity, unitPrice, status, metadata), totalItems, preparedItems (count where status = PREPARED).

Behavior: Find order with its items loaded (*-throws 404 if not found*). Count items by status (Prepared and not Prepared). Build the DTO with the full item list ordered by lineNumber ascending.

Test scenario:

- a) Create an order with 4 items: 2 PREPARED, 2 PENDING.
- b) GET /api/orders/{orderId}/details → verify DTO has totalItems=4, preparedItems=2, items ordered by lineNumber.
- c) Test with order that has no items → totalItems=0, preparedItems=0, empty items list.
- d) Test with non-existent order → 404.

9.4 Delivery Service Features (port 8084)

9.4.1 [S4-F1] Get Latest Delivery for an Order

Branch: feat/delivery/S4-F1/<ID>

Endpoint: GET /api/deliveries/order/{orderId}/latest

Behavior: Verify order exists (*-throws 404 if not found*). Query most recent *delivery* by updatedAt. *throws 404 if none*.

Test scenario:

- a) Create an order. Post 3 delivery updates with different timestamps.
- b) GET /api/deliveries/order/{orderId}/latest → should return the most recent delivery.
- c) Test with order that has no deliveries → 404.
- d) Test with non-existent order → 404.

9.4.2 [S4-F2] Create Delivery Record with Metadata (JSONB)

Branch: feat/delivery/S4-F2/<ID>

Endpoint: POST /api/deliveries/order/{orderId}

Request body: driverName, latitude, longitude, status, metadata map.

Behavior: Verify order exists *-throws 404 if not found*. Create Delivery with updatedAt = now. Save. Return *status code 201*.

Test scenario:

- a) Create an order.
- b) POST /api/deliveries/order/{orderId} with driverName="Hassan", coordinates, status=ASSIGNED, metadata={"estimatedArrival":25,"vehicleType":"motorcycle"} → 201.
- c) Verify the saved delivery has the correct orderId and metadata.
- d) Test with non-existent order → 404.

9.4.3 [S4-F3] Find Nearby Deliveries (DTO with Distance)

Branch: feat/delivery/S4-F3/<ID>

Endpoint: GET /api/deliveries/nearby?lat={lat}&lon={lon}&radiusKm={r}

Response DTO (NearbyDeliveryDTO): deliveryId, driverName, orderId, latitude, longitude, distanceKm.

Behavior: Find active deliveries (status IN ASSIGNED, PICKED_UP, IN_TRANSIT). Calculate distance (euclidean distance¹¹ * 111), filter by radius, sort ascending by distance. Map to DTO and return it.

Test scenario:

- a) Create 3 active deliveries: A at (30.04, 31.23), B at (30.05, 31.24), C at (31.00, 32.00) (far).
- b) GET /api/deliveries/nearby?lat=30.044&lon=31.235&radiusKm=5 → returns A and B (sorted by distance), not C.
- c) Test with radiusKm=0.1 → might return only the closest.

9.4.4 [S4-F4] Batch Delivery Updates (Transactional)

Branch: feat/delivery/S4-F4/<ID>

Endpoint: POST /api/deliveries/batch

Request body: orderId and list of delivery update objects.

Behavior (Transactional): Verify order *throws 404 if not found*. Validate coordinates *throws 400 if not valid, like Latitude must be between -90 and 90, Longitude must be between -180 and 180*. Save all delivery records. Return *status code 201 with count*.

Test scenario:

- a) Create an order.
- b) POST /api/deliveries/batch with orderId and 3 delivery update objects → 201 with count=3.
- c) Verify all 3 deliveries saved with correct orderId.
- d) Test with invalid coordinates (latitude=999) → 400.
- e) Test with non-existent order → 404.

9.4.5 [S4-F5] Filter Deliveries by Metadata (JSONB Query)

Branch: feat/delivery/S4-F5/<ID>

Endpoint: GET /api/deliveries/metadata/search?key={k}&operator={op}&value={v}

Operators: eq, gt, lt.

Behavior: Validate operator *throws 400 if not valid*. For gt/lt cast JSONB to numeric. Return matches.

Test scenario:

- a) Create deliveries with metadata: estimatedArrival=10, estimatedArrival=25, estimatedArrival=45.
- b) GET /api/deliveries/metadata/search?key=estimatedArrival&operator=gt&value=20 → returns 2 deliveries (25 and 45).
- c) GET /api/deliveries/metadata/search?key=estimatedArrival&operator=eq&value=10 → returns 1.
- d) GET /api/deliveries/metadata/search?key=estimatedArrival&operator=xyz&value=10 → 400.

¹¹euclidean distance = $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

9.4.6 [S4-F6] Order Delivery History in Date Range

Branch: feat/delivery/S4-F6/<ID>

Endpoint: GET /api/deliveries/order/{orderId}/history?startDate={d}&endDate={d}

Behavior: Verify order (*throws 404 if not found*). Create a date range query on updatedAt. Order by updatedAt ascending.

Test scenario:

- a) Create an order. Post 5 deliveries: 3 in March, 2 in February.
- b) GET /api/deliveries/order/{orderId}/history?startDate=2026-03-01&endDate=2026-03-31 → returns 3 deliveries ordered by updatedAt ascending.
- c) Test with non-existent order → 404.

9.4.7 [S4-F7] Purge Old Delivery Records (Transactional)

Branch: feat/delivery/S4-F7/<ID>

Endpoint: DELETE /api/deliveries/purge?olderThanDays={n}

Behavior (Transactional): Calculate cutoff (if the request says olderThanDays=30, then cutoff = today minus 30 days). Count then delete via native @Modifying query (only DELIVERED records). Return deleted count.

Test scenario:

- a) Create 10 deliveries: 7 with status DELIVERED and updatedAt 60 days ago, 3 with status IN_TRANSIT and updatedAt 60 days ago.
- b) DELETE /api/deliveries/purge?olderThanDays=30 → returns deletedCount=7 (only DELIVERED records).
- c) Verify the 3 IN_TRANSIT records remain.

9.4.8 [S4-F8] Delivery Performance Summary (DTO)

Branch: feat/delivery/S4-F8/<ID>

Endpoint: GET /api/deliveries/driver/{driverName}/summary?startDate={d}&endDate={d}

Response DTO (DeliveryPerformanceDTO): driverName, totalDeliveries, averageSpeed, maxSpeed, firstDelivery, lastDelivery.

Behavior: Query deliveries by driverName. Calculate the above metrics from JSONB speed field (cast to numeric). Build the DTO and return it. *Throw 404 if no records found for this driver.*

Test scenario:

- a) Create 5 deliveries by driver “Hassan” in March with metadata speeds: 20, 30, 40, 50, 0.
- b) GET /api/deliveries/driver/Hassan/summary?startDate=2026-03-01&endDate=2026-03-31 → totalDeliveries=5, averageSpeed=28, maxSpeed=50.
- c) Test with non-existent driver name → 404.

9.4.9 [S4-F9] Find Delayed Deliveries (JSONB + Cross-Table DTO)

Branch: feat/delivery/S4-F9/<ID>

Endpoint: GET /api/deliveries/delayed?maxEstimatedArrival={m}&sinceMinutes={min}

Response DTO (DelayedDeliveryDTO): deliveryId, driverName, orderId, latitude, longitude, estimatedArrival, updatedAt.

Behavior: Find active deliveries (status IN ASSIGNED, PICKED_UP, IN_TRANSIT) whose JSONB

`estimatedArrival` value > `maxEstimatedArrival` (cast to numeric) and whose `updatedAt` is within the last `sinceMinutes` minutes. These are deliveries taking longer than expected. Map results to DTOs.

Test scenario:

- a) Create 3 active deliveries with recent timestamps: A `estimatedArrival=15`, B `estimatedArrival=45`, C `estimatedArrival=60`.
- b) GET `/api/deliveries/delayed?maxEstimatedArrival=30&sinceMinutes=60` → returns B and C (`estimatedArrival > 30`).
- c) Test with `maxEstimatedArrival=100` → empty list.

9.5 Checkout Service Features (port 8085)

9.5.1 [S5-F1] Get Payments by Status and Date Range

Branch: `feat/checkout/S5-F1/<ID>`

Endpoint: GET `/api/payments/search?status={s}&startDate={d}&endDate={d}`

Behavior: Query with optional status and date range on `createdAt`. Return results with most recent first.

Test scenario:

- a) Create 5 payments: 2 COMPLETED in March, 1 REFUNDED in March, 2 COMPLETED in February.
- b) GET `/api/payments/search?status=COMPLETED&startDate=2026-03-01&endDate=2026-03-31` → returns 2, most recent first.
- c) GET `/api/payments/search?startDate=2026-03-01&endDate=2026-03-31` → returns 3 (no status filter).

9.5.2 [S5-F2] Process Refund (Transactional + JSONB Update)

Branch: `feat/checkout/S5-F2/<ID>`

Endpoint: PUT `/api/payments/{id}/refund`

Request body: `reason` (string).

Behavior (Transactional): Find payment –*throws 404 if not found*. Validate status COMPLETED –*throws 400 if not*. Set REFUNDED. Add `refundReason` and `refundedAt` to JSONB `transactionDetails`. Save and return the updated payment.

Test scenario:

- a) Create a COMPLETED payment.
- b) PUT `/api/payments/{id}/refund` with `{"reason":"wrong order delivered"}` → status=REFUNDED, `transactionDetails` has `refundReason` and `refundedAt`.
- c) Try refunding again → 400. Try refunding a FAILED payment → 400.

9.5.3 [S5-F3] User Payment Summary (DTO)

Branch: `feat/checkout/S5-F3/<ID>`

Endpoint: GET `/api/payments/user/{userId}/summary`

Response DTO (`UserPaymentSummaryDTO`): `userId`, `totalPayments`, `totalAmount`, `methodBreakdown` (a `Map<String, Double>`).

Behavior — step by step:

- a) **Verify user exists:** Check if the user exists –*throws 404 if not found*.
- b) **Query payment data grouped by method:** Query the payments of the user that have status COMPLETED and then group the results by the method of payment to compute the count and the sum of amount per each method.
- c) **Build the method breakdown map:** Populate a Map<String, Double> where each key is the payment method (e.g., "CREDIT_CARD") and the value is the total amount for that method.
- d) **Calculate totals:** totalPayments is the sum of all counts. totalAmount is the sum of all amounts.
- e) **Build and return the DTO.**

Test scenario:

- a) Create a user. Create 4 COMPLETED payments: 2 via CREDIT_CARD (80, 150), 1 via CASH_ON_DELIVERY (100), 1 via WALLET (70).
- b) GET /api/payments/user/{userId}/summary → totalPayments=4, totalAmount=400, method-Breakdown={"CREDIT_CARD":230,"CASH_ON_DELIVERY":100,"WALLET":70}.
- c) Test with non-existent user → 404.

9.5.4 [S5-F4] Process Payment for Order (Transactional)

Branch: feat/checkout/S5-F4/<ID>

Endpoint: POST /api/payments/order/{orderId}

Request body: method, cardLastFour (optional).

Behavior (Transactional): Verify order exists –*throws 404 if not found* and is DELIVERED –*throws 400 if having different status*. Check no COMPLETED payment exists for this order –*throws 400 if not with a message "already paid"*. Update the Payment that was created upon delivery with status PENDING, populate JSONB. Save. Return *status code 201* indicating successful payment.

Test scenario:

- a) Create a DELIVERED order.
- b) POST /api/payments/order/{orderId} with {"method":"CREDIT_CARD","cardLastFour":"4242"} → 201. Verify payment updated with correct amount and JSONB populated.
- c) Try paying again for the same order → 400 ("already paid"). Try paying for a PLACED order → 400.

9.5.5 [S5-F5] Apply Offer to Payment (Transactional + Join Entity)

Branch: feat/checkout/S5-F5/<ID>

Endpoint: POST /api/payments/{paymentId}/offers/{offerId}

No request body — both IDs come from the URL path.

Scenario: A customer has a pending payment of 150 EGP and wants to apply an offer code before checkout. The system validates the offer, calculates the discount, and creates a PaymentOffer join entity linking the two — the same pattern as OrderItem from Lab 4.

Behavior (Transactional) — step by step:

- a) **Find payment** by paymentId — *throw 404 if not found*.
- b) **Validate payment status:** The payment's status must be PENDING — *throw 400 with message "cannot apply offer to a completed/cancelled payment" if status is COMPLETED or REFUNDED*.

- c) **Find offer** by `offerId` — *throw 404 if not found.*
- d) **Validate offer is usable:**
 - Offer's `active` field must be `true` — *throw 400 if not.*
 - Offer's `expiryDate` must be in the future — *throw 400 if expired.*
 - Offer's `currentUses` must be less than `maxUses` — *throw 400 if limit reached.*
- e) **Check for duplicate:** Query `PaymentOffer` records — *throw 400 with message "offer already applied" if found.*
- f) **Calculate the discount amount** based on the offer's `discountType`:
 - If `PERCENTAGE`: `discount = payment.amount * offer.discountValue / 100`
Example: 150 EGP payment with a 30% offer → $150 * 30 / 100 = 45$ EGP discount.
 - If `FIXED`: `discount = offer.discountValue`
Example: 150 EGP payment with a 20 EGP fixed offer → 20 EGP discount.

Then **cap the discount**: if the calculated discount exceeds the payment amount, set it to the payment amount (an offer cannot produce a negative total).
- g) **Create the join entity:** Create a new `PaymentOffer` with the computed variables.
- h) **Update the offer:** Increment `currentUses` by 1.
- i) **Save** the `PaymentOffer` and the updated offer and payment.
- j) **Return** the updated payment with its list of applied offers.

Test scenario:

- a) Create a `PENDING` payment (`amount=150`). Create an offer: `code="HUNGRY30"`, `PERCENTAGE`, `discountValue=30`, `maxUses=3`.
- b) `POST /api/payments/{paymentId}/offers/{offerId}` → `PaymentOffer` created with `discountApplied=45`. Offer `currentUses` incremented to 1.
- c) Apply same offer again → 400. Create a `FIXED` offer with `discountValue=9999`, apply → `discountApplied` capped at 150.
- d) Test with expired offer → 400. Test with inactive offer → 400.
- e) Test applying offer to a `COMPLETED` payment → 400.

9.5.6 [S5-F6] Revenue Report by Date Range (Report DTO)

Branch: `feat/checkout/S5-F6/<ID>`

Endpoint: `GET /api/payments/reports/revenue?startDate={d}&endDate={d}`

Response DTO (`RevenueReportDTO`): `totalRevenue`, `totalTransactions`, `averagePayment`, `refundedAmount`, `refundCount`.

Scenario: An admin wants to see how much money the platform made in a given period, and how much was lost to refunds.

Behavior — step by step:

- a) **Validate dates:** *Throw 400 if startDate is after endDate.*
- b) **Query payments** in date range.
- c) **The five metrics:**

- `totalRevenue` — Sum of amount where `status = 'COMPLETED'`.
- `totalTransactions` — Count where `status = 'COMPLETED'`.
- `averagePayment` — `totalRevenue / totalTransactions`. Handle division by zero.
- `refundedAmount` — Sum of amount where `status = 'REFUNDED'`.
- `refundCount` — Count where `status = 'REFUNDED'`.

d) **Build and return the DTO.**

Test scenario:

- Create payments in March: 5 COMPLETED (amounts: 80,100,120,150,200 = total 650), 2 REFUNDED (amounts: 60,90 = total 150).
- GET `/api/payments/reports/revenue?startDate=2026-03-01&endDate=2026-03-31` → `totalRevenue=650, totalTransactions=5, averagePayment=130, refundedAmount=150, refundCount=2`.
- Test with `startDate` after `endDate` → 400.

9.5.7 [S5-F7] Retry Failed Payment (Transactional)

Branch: `feat/checkout/S5-F7/<ID>`

Endpoint: PUT `/api/payments/{id}/retry`

No request body — the payment ID comes from the URL path.

Scenario: A payment failed (e.g., card declined). The customer wants to try again. Instead of creating a new payment, the system retries the existing one.

Behavior (Transactional) — step by step:

- Find payment** by id — *throw 404 if not found*.
- Validate status:** Must be FAILED — *throw 400 if anything else*.
- Update status:** Set to COMPLETED.
- Update JSONB transactionDetails:**
 - `retryAttempt` — increment by 1.
 - `gatewayResponse` — overwrite with "approved".

So if the JSONB was:

```
{
  "gatewayResponse":"declined",
  "cardLastFour":"4242",
  "retryAttempt":0,
  "failureReason":"insufficient funds"
}
```

After retry it becomes:

```
{
  "gatewayResponse":"approved",
  "cardLastFour":"4242",
  "retryAttempt":1,
  "failureReason":"insufficient funds"
}
```

- Save** the payment.

- f) **Return** the updated payment entity.

Test scenario:

- a) Create a FAILED payment with transactionDetails {"gatewayResponse": "declined", "retryAttempt": 0}.
- b) PUT /api/payments/{id}/retry → status=COMPLETED, retryAttempt=1, gatewayResponse="approved".
- c) Try retrying a COMPLETED payment → 400. Try retrying a REFUNDED payment → 400.

9.5.8 [S5-F8] Get Payment Details with Applied Offers (Join Entity DTO)

Branch: feat/checkout/S5-F8/<ID>

Endpoint: GET /api/payments/{paymentId}/details

Response DTO (PaymentDetailsDTO): paymentId, orderId, userId, originalAmount, method, status, transactionDetails (JSONB), appliedOffers (list of objects, each with offerCode, discountType, discountApplied, appliedAt), totalDiscount (sum of all discounts), finalAmount (originalAmount - totalDiscount).

Behavior: Find payment with its PaymentOffer entries loaded (*-throws 404 if not found*). For each PaymentOffer, access the linked Offer entity to get the offer code and discount type. Calculate totalDiscount and finalAmount. Build the DTO and return it.

Test scenario:

- a) Create a payment (amount=150). Apply 2 offers: discountApplied=45 and discountApplied=20.
- b) GET /api/payments/{paymentId}/details → originalAmount=150, appliedOffers has 2 entries, totalDiscount=65, finalAmount=85.
- c) Test with payment that has no offers → totalDiscount=0, finalAmount=originalAmount.
- d) Test with non-existent payment → 404.

9.5.9 [S5-F9] Get Most Used Offers Report (Join Entity + Aggregation)

Branch: feat/checkout/S5-F9/<ID>

Endpoint: GET /api/payments/offers/top-used?limit={n}

Response DTO (OfferUsageDTO): offerId, code, discountType, discountValue, timesUsed, totalDiscountGiven, active, expired.

Scenario: A platform manager wants to see which offers are performing best.

Behavior — step by step:

- a) **Write a query** that searches for offers with payment_offers. Group by offer. For each offer, compute:
 - timesUsed — the offer's currentUses value.
 - totalDiscountGiven — SUM(payment_offers.discount_applied). This is the total money saved by customers who used this offer.

For example, if offer "HUNGRY30" was applied to 3 payments with discounts of 45, 45, and 30 EGP, then timesUsed = 3 and totalDiscountGiven = 120.0.
- b) **Order by** timesUsed descending and **limit** to n results.
- c) **Map each result to an OfferUsageDTO:** Populate all fields. expired is a computed boolean in Java.
- d) **Return** the list of DTOs.

Test scenario:

- a) Create 3 offers. Apply offer A to 5 payments (total discount=225), offer B to 3 payments (total=90), offer C to 1 payment (total=20).
- b) GET /api/payments/offers/top-used?limit=2 → returns offer A (timesUsed=5, totalDiscount-Given=225) then offer B.
- c) Verify “expired” field is true for offers past their expiryDate.

10 Git Workflow — Feature Development

Every team member follows this workflow for **every feature**:

- a) Start from main: `git checkout main && git pull origin main`
- b) Create feature branch: `git checkout -b feat/<service>/<feature-ID>/<YOUR-STUDENT-ID>`
- c) Implement: repository methods → service logic → controller endpoint → test via Postman.
- d) Commit: `git commit -m "feat(<service-name>): <description> (<YOUR-STUDENT-ID>)"` (except only the first commit which is the initialization of the project)
- e) Push and create a Pull Request on GitHub.
- f) At least 1 teammate reviews and approves.
- g) **Merge into main using a regular merge commit (“Create a merge commit” on GitHub). Do NOT use squash merge.** A regular merge preserves the branch name in the merge commit message (e.g., Merge pull request #12 from feat/user/S1-F1/55-8078), which the auto-grader uses to verify who implemented which feature.
- h) **Do NOT delete the feature branch after merging.** The auto-grader will verify that each feature has a correctly-named branch containing the student ID. If you delete the branch, the grader will not be able to verify it and this may result in deductions in your grade.

11 Dockerization (Phase D)

- a) Create a **Dockerfile** inside each service module using `eclipse-temurin:25.0.2_10-jdk`. Copy that service’s JAR and expose port 8080.
- b) Update the root `docker-compose.yaml` to include all 5 services alongside PostgreSQL. Each service:
 - Builds from its own Dockerfile (e.g., `build: ./user-service`)
 - Maps its host port to container port 8080 (e.g., `8081:8080`)
 - Overrides datasource URL to `jdbc:postgresql://postgres:5432/fooddeliverydb`
 - Depends on the PostgreSQL service
- c) Build: `mvn clean package -DskipTests` from the root.
- d) Run: `docker compose up --build`
- e) Verify all services respond:
 - `localhost:8081/api/users` (User Service)
 - `localhost:8082/api/restaurants` (Restaurant Service)
 - `localhost:8083/api/orders` (Order Service)
 - `localhost:8084/api/deliveries` (Delivery Service)
 - `localhost:8085/api/payments` (Checkout Service)
- f) Git: branch `feat/docker/<ID>`, commit with ID, PR, merge.

12 Work Distribution (Recommended):

Each member implements around 3 features. Phase A (all entities + CRUD for every entity) is done collectively (You can divide it across the sub_team that will implement the service or divide it across all the members equally as you wish).

13 Submission & Auto-Grader

- a) Repository must be **private**.
- b) **Scalable-Submissions** must be added as a collaborator.
- c) **team.json** must be present and valid at the project root.
- d) **docker compose up** must start PostgreSQL + all 5 services.
- e) Services must respond at **localhost:8081** through **localhost:8085**.
- f) Git history must show feature branches merged via PRs with student IDs.
- g) Commits must follow **feat(<service>): <desc> (<ID>)** format.
- h) Details about the usage of the auto grader and the testing will be sent later

Important: Grading is **per member**. A feature only counts for the member whose GitHub username authored the commits on a branch containing their student ID. Features with mismatched or missing IDs receive zero credit. (The zero will affect both, the team member and the whole team). The only case that the team member will receive a different grade from the rest of the team is if there is no history of commits that shows that this member worked in the milestone (if his/her work was done then the whole team will receive their grade fully and he/she will receive zero. If his/her work was missing from the overall work, the whole team will be deducted for the missing work and he/she will still receive zero). It is the responsibility of the scrum master to make sure that everyone is working and the whole requirements are being delivered, if there is an issue with any member(s) then a form will be sent for complains that you can fill it

Deadline: The Milestone deadline is Saturday 11/04/2026 at 11:59 PM

Submission: Submission form will be sent later to the scrum masters.

14 Appendix: JSONB Sample Data

Below is one example per service showing what the JSONB columns look like in practice. A full database seeder will be provided separately.

User.preferences:

```
{
  "language": "en",
  "notifications": {"email": true, "sms": false},
  "defaultPaymentMethod": "WALLET",
  "dietaryRestrictions": "vegetarian"
}
```

Restaurant.details:

```
{
  "address": "12 Zamalek Street, Cairo",
  "openingHours": "10AM-12AM",
}
```

```
"deliveryRadius": 8,  
"minOrder": 50,  
"avgPrepTime": 25,  
"licenseNumber": "REST-2024-4567"  
}
```

Order.metadata:

```
{  
  "specialInstructions": "no onions, extra sauce",  
  "cutleryRequested": false,  
  "estimatedDeliveryTime": 35,  
  "deliveryType": "express"  
}
```

Delivery.metadata:

```
{  
  "estimatedArrival": 25,  
  "distanceRemaining": 3.5,  
  "vehicleType": "motorcycle",  
  "speed": 35.0,  
  "contactPhone": "+201012345678"  
}
```

Payment.transactionDetails:

```
{
  "gatewayResponse": "approved",
  "cardLastFour": "4242",
  "receiptUrl": "https://receipts.example.com/abc",
  "failureReason": null,
  "tipAmount": 15.0
}
```

15 Appendix: Example Database Tables

The following tables show what your database should look like after seeding some test data. Use these as a reference when testing your CRUD operations and features via Postman. All tables coexist in the same `talabatdb` database.

Note: JSONB columns are shown as formatted JSON code blocks below each table for readability. In the actual database, each block is stored as a single JSONB column value in the corresponding row.

15.1 User Service Tables

15.1.1 users table

id	name	email	password	phone	role	status	created_at
1	Ahmed Hassan	ahmed@mail.com	pass123	+201011111111	CUSTOMER	ACTIVE	2026-03-01 10:00
2	Sara Mohamed	sara@mail.com	pass456	+201022222222	CUSTOMER	ACTIVE	2026-03-02 14:30
3	Admin User	admin@food.com	admin789	+201033333333	ADMIN	ACTIVE	2026-03-01 08:00

JSONB column — preferences:

User 1 (Ahmed):

```
{
  "language": "ar",
  "notifications": {"email": true, "sms": false},
  "defaultPaymentMethod": "CREDIT_CARD",
  "dietaryRestrictions": "none"
}
```

User 2 (Sara):

```
{
  "language": "en",
  "notifications": {"email": true, "sms": true},
  "defaultPaymentMethod": "WALLET",
  "dietaryRestrictions": "vegetarian"
}
```

User 3 (Admin):

```
{
  "language": "en",
  "notifications": {"email": true, "sms": false},
  "defaultPaymentMethod": "CREDIT_CARD"
}
```

15.1.2 delivery_addresses table

id	user_id	label	street_address	city	lat	lon	is_default
1	1	Home	15 Nile St, Zamalek	Cairo	30.060	31.220	true
2	1	Office	42 Smart Village, B5	Giza	30.071	31.013	false
3	2	Home	8 Corniche Rd, Apt 3B	Alexandria	31.215	29.955	true

JSONB column — metadata:

Address 1 (Ahmed -- Home):

```
{
  "floor": 5,
  "landmark": "Next to Nile Pharmacy",
  "deliveryInstructions": "Ring twice",
  "phoneForDelivery": "+20101111111"
}
```

Address 2 (Ahmed -- Office):

```
{
  "floor": 2,
  "gateCode": "7788",
  "landmark": "Building B5 entrance",
  "deliveryInstructions": "Ask for Ahmed at reception"
}
```

Address 3 (Sara -- Home):

```
{
  "floor": 3,
  "landmark": "Blue building opposite the mosque",
  "deliveryInstructions": "Leave with doorman"
}
```

15.2 Restaurant Service Tables

15.2.1 restaurants table

id	name	cuisine_type	status	phone	rating	total_ratings
1	Koshary El Tahrir	EGYPTIAN	OPEN	+20221111111	4.7	3
2	Pizza Roma	ITALIAN	OPEN	+20222222222	4.2	2
3	Dragon Palace	CHINESE	CLOSED	+20223333333	3.9	1

(email column omitted for space, createdAt: Restaurant 1 = 2026-01-15, Restaurant 2 = 2026-02-20)

JSONB column — details:

Restaurant 1 (Koshary El Tahrir):

```
{
  "address": "12 Tahrir Square, Cairo",
  "openingHours": "8AM-12AM",
  "deliveryRadius": 10,
  "minOrder": 30,
  "avgPrepTime": 15,
  "licenseNumber": "REST-2024-1001"
}
```

Restaurant 2 (Pizza Roma):

```
{
  "address": "45 Zamalek St, Cairo",
  "openingHours": "11AM-11PM",
  "deliveryRadius": 8,
  "minOrder": 80,
  "avgPrepTime": 30,
  "licenseNumber": "REST-2024-2002"
}
```

Restaurant 3 (Dragon Palace):

```
{
  "address": "99 Maadi Ring Rd",
  "openingHours": "12PM-10PM",
  "deliveryRadius": 5,
  "minOrder": 100,
  "avgPrepTime": 25,
  "licenseNumber": "REST-2024-3003"
}
```

15.2.2 menu_items table

id	restaurant_id	name	price	category	available
1	1	Koshary Regular	35	MAIN	true
2	1	Koshary Special	55	MAIN	true
3	1	Lentil Soup	20	APPETIZER	false
4	2	Margherita Pizza	120	MAIN	true
5	2	Pepperoni Pizza	150	MAIN	true
6	2	Tiramisu	80	DESSERT	true

(description column omitted for space, createdAt: Items 1–3 = 2026-01-15, Item 4 = 2026-02-20)

JSONB column — metadata:

Item 1 (Koshary Regular):

```
{
  "calories": 450,
  "allergens": ["gluten"],
  "prepTime": 10,
  "spiceLevel": "mild",
  "dietaryTags": ["vegetarian"]
}
```

Item 4 (Margherita Pizza):

```
{
  "calories": 680,
  "allergens": ["gluten", "dairy"],
  "prepTime": 20,
  "spiceLevel": "none",
  "dietaryTags": ["vegetarian"]
}
```

Item 5 (Pepperoni Pizza):

```
{
```



```

    "calories": 780,
    "allergens": ["gluten", "dairy"],
    "prepTime": 20,
    "spiceLevel": "mild",
    "dietaryTags": []
  }

```

15.3 Order Service Tables

15.3.1 orders table

id	user_id	restaurant_id	status	total_amount	order_date	delivered_at
1	1	1	DELIVERED	125.00	2026-03-10 12:30	2026-03-10 13:05
2	2	2	CONFIRMED	350.00	2026-03-18 19:00	<i>null</i>
3	1	2	CANCELLED	150.00	2026-03-19 20:00	<i>null</i>

JSONB column — metadata:

Order 1 (Ahmed -- Delivered from Koshary):

```

{
  "specialInstructions": "extra sauce on the side",
  "cutleryRequested": false,
  "estimatedDeliveryTime": 25,
  "deliveryType": "standard"
}

```

Order 2 (Sara -- Confirmed from Pizza Roma):

```

{
  "specialInstructions": "no onions on pizza",
  "cutleryRequested": true,
  "estimatedDeliveryTime": 40,
  "deliveryType": "express"
}

```

Order 3 (Ahmed -- Cancelled from Pizza Roma):

```

{
  "deliveryType": "standard",
  "cancellationReason": "Restaurant too far"
}

```

15.3.2 order_items table

id	order_id	line	item_name	menu_item_id	qty	unit_price	status
1	1	1	Koshary Regular	1	2	35.00	PREPARED
2	1	2	Koshary Special	2	1	55.00	PREPARED
3	2	1	Margherita Pizza	4	1	120.00	PENDING
4	2	2	Pepperoni Pizza	5	1	150.00	PENDING
5	2	3	Tiramisu	6	1	80.00	PENDING
6	3	1	Pepperoni Pizza	5	1	150.00	CANCELLED

(*line* = *lineNumber*, *qty* = *quantity*)

JSONB column — metadata:

Item 1 (Koshary Regular in Order 1):

```
{
  "customizations": ["extra lentils", "hot sauce"],
  "size": "regular",
  "notes": "one for me, one for a friend"
}
```

Item 3 (Margherita in Order 2):

```
{
  "customizations": ["no onions", "extra cheese"],
  "size": "large"
}
```

Item 5 (Tiramisu in Order 2):

```
{
  "customizations": [],
  "notes": "birthday candle please"
}
```

Reading the data: Order #1 was placed by Ahmed from Koshary El Tahrir — he ordered 2 regular koshary and 1 special for a total of 125 EGP. The order was delivered. Order #2 was placed by Sara from Pizza Roma — she is ordering a Margherita, a Pepperoni, and a Tiramisu. Order #3 was cancelled by Ahmed before the restaurant started preparing it.

15.4 Delivery Service Tables

15.4.1 deliveries table

id	order_id	driver_name	status	lat	lon	updated_at
1	1	Hassan	DELIVERED	30.060	31.220	2026-03-10 13:05
2	2	Omar	PICKED_UP	30.058	31.222	2026-03-18 19:30

JSONB column — metadata:

Delivery 1 (Hassan -- Order 1, Delivered):

```
{
  "estimatedArrival": 20,
  "distanceRemaining": 0,
  "vehicleType": "motorcycle",
  "speed": 0,
  "contactPhone": "+201055551111"
}
```

Delivery 2 (Omar -- Order 2, Picked Up):

```
{
  "estimatedArrival": 25,
  "distanceRemaining": 3.2,
  "vehicleType": "motorcycle",
  "speed": 35.0,
  "contactPhone": "+201055552222"
}
```

Reading the data: Delivery #1 is for Ahmed's koshary order — driver Hassan has already delivered it (speed=0, distance=0). Delivery #2 is for Sara's pizza order — driver Omar has picked up the food

and is 3.2 km away from Sara, currently moving at 35 km/h. There is no delivery for Order #3 because it was cancelled.

15.5 Checkout Service Tables

15.5.1 payments table

id	order_id	user_id	amount	method	status	created_at
1	1	1	125.00	CREDIT_CARD	COMPLETED	2026-03-10 13:10
2	2	2	350.00	WALLET	PENDING	2026-03-18 19:05
3	3	1	150.00	CREDIT_CARD	REFUNDED	2026-03-19 20:10

JSONB column — transactionDetails:

Payment 1 (Ahmed -- Completed):

```
{
  "gatewayResponse": "approved",
  "cardLastFour": "4242",
  "receiptUrl": "https://pay.example/food001",
  "tipAmount": 10.0
}
```

Payment 2 (Sara -- Pending):

```
{
  "gatewayResponse": null,
  "walletBalance": 5000,
  "tipAmount": 15.0
}
```

Payment 3 (Ahmed -- Refunded):

```
{
  "gatewayResponse": "approved",
  "cardLastFour": "9876",
  "refundReason": "Order cancelled",
  "refundedAt": "2026-03-19T20:30"
}
```

15.5.2 offers table

id	code	discount_type	value	max	used	expiry_date	active
1	HUNGRY30	PERCENTAGE	30.0	50	2	2026-06-30 23:59	true
2	FLAT20	FIXED	20.0	100	1	2026-04-30 23:59	true
3	OLDOFFER	PERCENTAGE	15.0	10	10	2026-02-28 23:59	false

(Column headers shortened: *value* = *discountValue*, *max* = *maxUses*, *used* = *currentUses*)

JSONB column — metadata:

Offer 1 (HUNGRY30):

```
{
  "eligibleCuisines": ["EGYPTIAN", "ITALIAN"],
  "minimumOrder": 50,
  "termsAndConditions": "New customers only"
}
```

Offer 2 (FLAT20):

```
{
  "eligibleCuisines": ["FAST_FOOD"],
  "minimumOrder": 80,
  "applicableRestaurants": ["Burger King", "McDonalds"]
}
```

Offer 3 (OLDOFFER):

```
{
  "eligibleCuisines": ["CHINESE", "SEAFOOD"],
  "minimumOrder": 100
}
```

15.5.3 payment_offers table (Join Entity)

id	payment_id	offer_id	discount_applied	applied_at
1	1	1	37.50	2026-03-10 13:09
2	2	1	105.00	2026-03-18 19:04
3	2	2	20.00	2026-03-18 19:04

Reading the data:

- Payment #1 (Ahmed's koshary order, 125 EGP) had offer HUNGRY30 applied — 30% of 125 = 37.50 EGP discount.
- Payment #2 (Sara's pizza order, 350 EGP) had **two** offers applied: HUNGRY30 (30% = 105 EGP) and FLAT20 (fixed 20 EGP). Total discount = 125 EGP.
- Payment #3 (Ahmed's cancelled order) had no offers applied.
- Offer HUNGRY30 has `current_uses` = 2 because it was used in payments #1 and #2.
- Offer OLDOFFER is inactive because it has reached its max uses (10/10) and its expiry date has passed.

15.6 How the Tables Connect (Cross-Service)

Since all services share one database, you can trace data across tables:

- a) **Ahmed** (user 1) placed **Order #1** from **Koshary El Tahrir** (restaurant 1), ordering 2 regular koshary and 1 special.
- b) The order was assigned to driver **Hassan** via **Delivery #1**, who picked it up and delivered it to Ahmed's home address.
- c) Ahmed paid via **Payment #1** (CREDIT_CARD, 125 EGP) with offer HUNGRY30 applied (saved 37.50 EGP). He also tipped 10 EGP.
- d) The order was delivered at 13:05 — both the order status and delivery status show DELIVERED.
- e) A native SQL query in the **Order Service** can JOIN `orders` with `users` (`user_id`), `restaurants` (`restaurant_id`), `deliveries` (`order_id`), and `payments` (`order_id`) to build a complete order history DTO.

This is the kind of cross-service data access you will implement using **native SQL JOINS** on the shared database.